

Documentation: CI Pipeline Setup for Java + TestNG + Allure on GitHub Actions

1. Overview

The project is a Java-based Test Automation Framework using:

- **Maven**
- **TestNG**
- **Selenium**
- **Allure Reporting**

The goal was to automate:

- Running tests on every push / PR
 - Generating **Allure reports**
 - Uploading reports as build artifacts
 - Ensuring tests run in GitHub's CI environment (Ubuntu runner)
-

2. What Problem We Solved

Initially:

- Local tests were working
- But GitHub Actions failed due to:
 - Missing JDK
 - Missing Maven cache
 - Allure plugin not found
 - Allure results folder not created inside `/target`
 - CLI-based Allure installation not available
 - Docker-based Allure plugins breaking the run

You learned **why** each failure happened and fixed it step by step.

3. What You Implemented (Step by Step)

Step 1: Created GitHub Actions Workflow

Created file:

```
.github/workflows/health_check.yml
```

Configured triggers:

```
on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]
```

Meaning:

- Every push to `main` runs the tests
 - Every PR targeting `main` also runs tests
-

Step 2: Checked Out the Repository

This step loads the source code into GitHub runner.

```
- name: Checkout Repository
  uses: actions/checkout@v4
```

Step 3: Installed the Correct Java Version

Your framework uses Java 23, so:

```
- name: Set up JDK
  uses: actions/setup-java@v4
  with:
    distribution: 'temurin'
    java-version: '23'
```

GitHub runner now has:

- Java compiler
 - Java runtime
 - Needed for Maven + TestNG
-

Step 4: Cached Maven Dependencies

This **speeds up CI** by using previous Maven downloads.

```
- name: Cache Maven Dependencies
  uses: actions/cache@v3
  with:
    path: ~/.m2
```

```
key: ${{ runner.os }}-m2-${{ hashFiles('**/pom.xml') }}
```

This saves the .m2 folder between workflow runs.

Step 5: Ran Maven Tests

This is the core testing step.

```
- name: Run Maven Tests
  run: mvn clean test
```

This:

- Cleans previous build files
 - Compiles code
 - Runs all TestNG tests
 - Generates allure-results inside target/
-

Step 6: Uploaded Allure Results

These are raw JSON files.

```
- name: Upload Allure Results
  uses: actions/upload-artifact@v4
  with:
    name: allure-results
    path: target/allure-results
```

This lets you download test results from GitHub UI.

Step 7: Installed Allure CLI

Since GitHub runner did NOT have Allure CLI installed, we manually installed it:

```
- name: Install Allure CLI
  run: |
    sudo apt update
    sudo apt install -y allure
```

Step 8: Generated Allure HTML Report

This converts JSON → HTML UI.

```
- name: Generate Allure Report
  run: |
    allure generate target/allure-results --clean -o allure-report
```

Step 9: Upload Final HTML Report

Uploaded the HTML folder as artifact:

```
- name: Upload Allure HTML Report
  uses: actions/upload-artifact@v4
  with:
    name: allure-report
    path: allure-report
```

Now you can **download and view the report** directly from GitHub Actions.

4. What You Learned Technically

- ✓ How GitHub Actions workflows are structured
 - ✓ How CI pipelines run on cloud runners
 - ✓ How Maven, Java, TestNG behave inside CI
 - ✓ How caching works in CI
 - ✓ How to install dependencies using Linux commands
 - ✓ How Allure CLI works (and why plugin alone isn't enough)
 - ✓ How to upload artifacts for reports
 - ✓ How to debug YAML errors
 - ✓ How Docker-based actions behave (and why you removed them)
-

5. PM Interview Angle: What You Can Claim

You effectively built a CI pipeline:

1 Automated Build + Test Execution

- Eliminated manual test execution
- Ensured all code commits are validated

2 Shift-Left Quality

- Provided immediate feedback on failures
- Enabled faster development cycles

3 Improved Developer Experience

- Maven caching improved speed
- Allure reports improved test visibility

4 Removed Infrastructure Issues

- Ensured Java + Maven setup works consistently

5 Improved Observability

- Stored test artifacts
- Stored HTML reports
- Clearer failure logs

6 Release Confidence Boost

Teams get visual reports for each change → safer deployments.

6. Summary (1-Minute PM Explanation)

“I automated the CI pipeline for our Java/TestNG automation suite using GitHub Actions. The pipeline compiles code, runs the full test suite, generates Allure reports, caches Maven dependencies, and uploads test artifacts automatically on every push and pull request. This ensures consistent test execution in a clean cloud environment, with detailed HTML reports for traceability. Overall it increased reliability, reduced manual effort, and improved release confidence.”
